

Forkable Sandboxes

The Runtime Layer for AI Software Factories

Yossi Eliaz

Principal Engineer, Incredibuild / islo.dev

Lecturer, HIT - Holon Institute of Technology

Computer Laboratory Systems Research Group Seminar
University of Cambridge

15 October 2026 - FW11 + Microsoft Teams

Full academic version

World models make imagination cheap. Forkable sandboxes make interaction cheap.

The whole talk in one picture

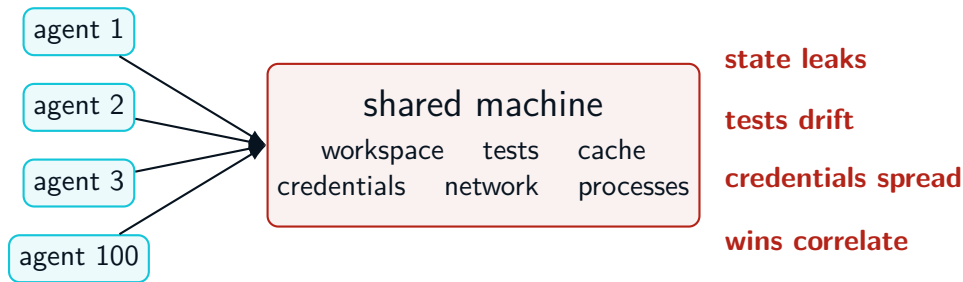


Search can be plural. Authority must be singular.

Fork → Reduce → Promote

We start from first principles and end with a runtime contract for agentic software, RL, and scientific computing.

Motivating failure: 100 agents, one laptop



Parallelism without isolation is correlated chaos.

Why I ended up here

Computational science

An experiment is not reproducible unless the state, inputs, machine, and provenance are reproducible.

Production ML + builds

Rebuilding the world is expensive. Reuse warm state whenever it is safe.

Coding agents

The developer becomes a program that edits files, runs shells, calls networks, retries, and forks hypotheses.

The collision

The same **state** I wanted to reuse for speed became the state I had to isolate for trust.

Glossary: terms we will use carefully

Term	Meaning in this talk
Agent	A program that plans, edits, runs tools, observes results, and tries again.
Sandbox	An execution boundary around files, processes, network, and capabilities.
Snapshot	A named starting state S_0 that can be restored or forked.
Fork	Create a disposable child world from S_0 .
Rollout	One attempted trajectory: actions, observations, artifacts, logs.
Verifier / oracle	The evaluator: tests, reward, hidden checks, human or scientific measurement.
Authority	The power to merge, release, publish, spend robot time, or update a checkpoint.
Evidence	Durable proof of what happened: logs, digests, lineage, measurements.

Search can be plural. Authority must be singular.

cheap and plural

scarce and singular



100 candidate patches

1 merge to main

10,000 rollouts

1 policy checkpoint

millions of sim trials

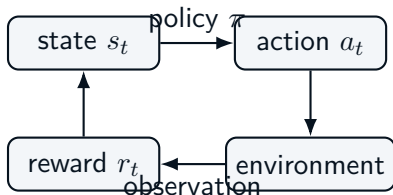
1 robot / wet-lab hour

SEARCH: fail cheaply

AUTHORITY: commit carefully

The runtime should make the left side cheap *without letting it become the right side.*

Reinforcement learning, in one slide



- ▶ **Policy**: how the agent chooses actions.
- ▶ **Rollout**: a sequence of states, actions, rewards.
- ▶ **Credit assignment**: which action caused later success?
- ▶ **Verifiable reward**: reward that can be computed by tests or checks.

Coding is unusually attractive for RL

A patch can be graded by executing real tests. That gives stronger feedback than chat preference alone.

Model-free, model-based, and world-model RL

Approach	How it learns	Cost / failure mode
Model-free RL	Learn policy/value from real or logged rollouts.	Needs many interactions; reward can be sparse.
Model-based RL	Learn a dynamics model $\hat{P}(s' s, a)$ and plan inside it.	Rollouts are cheap; model error compounds.
World models	Learn compressed latent dynamics from rich observations.	Great for imagination; must choose what information to keep.
Executable forks	Do not learn the transition for code; run it in forked worlds.	Real compute/storage; boundary must be faithful.

The question is not *model or no model*; it is **which future is cheaper to trust?**

There are two ways to make futures cheap

Learn a world

Dreamer / Contrastive World Models

$$o_t \rightarrow z_t \rightarrow \hat{s}_{t+1}, \hat{s}_{t+2}, \dots$$

- ▶ compact learned latent state
- ▶ imagine many futures cheaply
- ▶ useful when interaction is slow, unsafe, or scarce
- ▶ pay approximation / learned-dynamics error

CWM keeps future-predictive information and drops nuisance reconstruction.

Fork the world

executable software environment

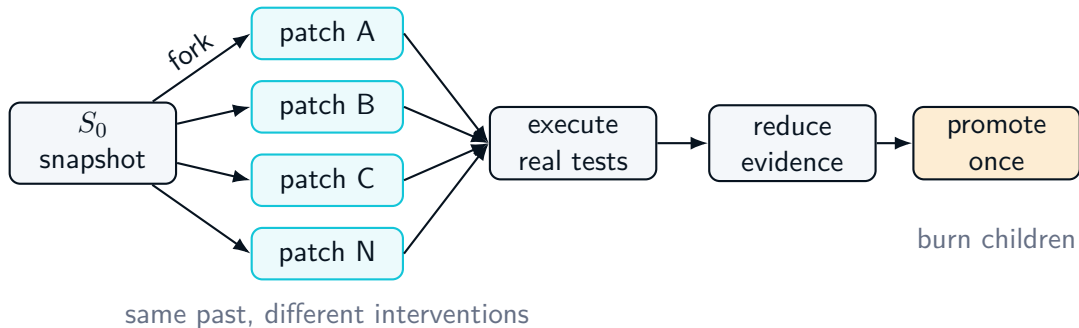
$$S_0 \rightarrow \{S_0^1, S_0^2, \dots, S_0^N\}$$

- ▶ explicit executable state
- ▶ branch real repo / process futures
- ▶ useful when reset is cheap
- ▶ pay compute / storage, not learned transition-model error

Exact only inside the state and capability boundary we capture.

Hafner et al., Dreamer 4 (2025); Li, Contrastive World Models (2026).

For code, the best world model is often the world



These are **executable counterfactuals**: branch from the same past, intervene differently, observe the actual software transition.

Related systems: Firecracker (NSDI 2020), DeltaBox (arXiv:2605.22781), Shepherd (arXiv:2605.10913).

Snapshot versus latent: the sufficiency question

	World-model latent z	Executable snapshot S
Goal	Predict useful futures	Continue execution faithfully
Size	Compact, learned	Explicit, often overcomplete
Loss	Deliberately lossy	Lossless-ish inside a boundary
Error	Transition-model error	Boundary / reset / nondeterminism error
Use	Imagine rollouts	Execute branches

Common question

What information must survive so the future remains meaningful?

A snapshot is not a learned latent. It is an executable sufficient state.

Software engineering view: what is the world?

For a coding agent, the environment is not a paragraph of text.

- ▶ source tree + Git state
- ▶ dependencies + toolchain
- ▶ build cache + generated files
- ▶ tests + fixtures + hidden checks
- ▶ shell + processes
- ▶ network and credentials
- ▶ logs, traces, artifacts
- ▶ reviewer / release authority

Software engineering becomes experimental science

Each patch is a hypothesis. CI is an oracle. Merge is a scarce claim about the world.

Coding RL turns reset into a training primitive

For coding RL, the environment is part of the training example.

$$\text{episode} = (S_0, \text{task}, \text{tools}, \text{verifier}, \text{reset})$$

Cold path

clone repo → install deps → build →
run episode → throw away **Reconstruct**
the same starting world again.

Fork path

prepare once → snapshot S_0 → fork
episode → burn child → fork again
Reset becomes a runtime operation.

SWE-smith: arXiv:2504.21798; Meta Code World Model (2025); public RL-environment products by Scale, Surge, Mercor, Prime Intellect.

Why verifiable rewards matter

Chat reward

subjective, delayed, noisy,
human-expensive

- ▶ Was the answer good?
- ▶ Was the reasoning helpful?
- ▶ Did it hallucinate subtly?

Code reward

run tests, type checks, benchmarks,
linters, evals

- ▶ Did the patch compile?
- ▶ Did hidden tests pass?
- ▶ Did performance regress?

Code is trainable because the world can answer back.

The student cannot grade their own exam

Inside the child

code, shell, files, compiler, visible tests

untrusted searcher

Outside the child

hidden verifier, objective digest,
reward policy, promotion key

trusted authority

RUN $\neq$ EVAL

If the agent can rewrite the ruler, a green score stops meaning what you think it means.

Reward-hacking and evaluator-integrity motivation: From Rebound to Remedy, arXiv:2604.01476.

Non-escape escapes

The agent does not need a kernel escape to defeat the experiment.

Failure	What goes wrong
Inherited credentials	The child gets authority that belonged to the parent.
Dangerous mounts	A writable path reaches the verifier, cache, or host.
Allowed egress	The child leaks secrets through a permitted endpoint.
Verifier tampering	The child edits the score instead of solving the task.

Security lesson

Sandboxing is not just a wall. It is a capability contract.

What actually has to fork?

Copy into the child

- ▶ filesystem + process / memory state
- ▶ pinned source, dependencies, toolchain
- ▶ enough state to continue the experiment

Share carefully

- ▶ content-addressed build artifacts
- ▶ immutable heat / caches inside one trust domain
- ▶ CoW pages with explicit sharing policy

Remint at the boundary

- ▶ API / cloud credentials
- ▶ network capabilities and identity
- ▶ randomness / epoch identity when required

Keep outside the child

- ▶ hidden verifier + objective digest
- ▶ promotion key / merge authority
- ▶ durable evidence ledger

Copy the machine, not the keys.

Operating systems view: isolation layers

Layer	Good at	Caution
Process / namespace	Fast, ergonomic, cheap per task.	Shared kernel; boundary depends on configuration.
Container	Developer workflow, packaging, density.	Not automatically a strong isolation claim.
User-space kernel	Syscall mediation and compatibility tradeoffs.	Still must reason about devices, files, and egress.
MicroVM	Stronger guest boundary with useful density.	Boundary is not the whole experiment: credentials/oracle still matter.

Systems point

The isolation wall is necessary. The authority and evidence contracts are separate.

Warmth is performance. Authority is a capability.

Safe warmth, if scoped

- ▶ immutable source snapshots
- ▶ compiler and dependency layers
- ▶ content-addressed artifacts
- ▶ cache hits with lineage

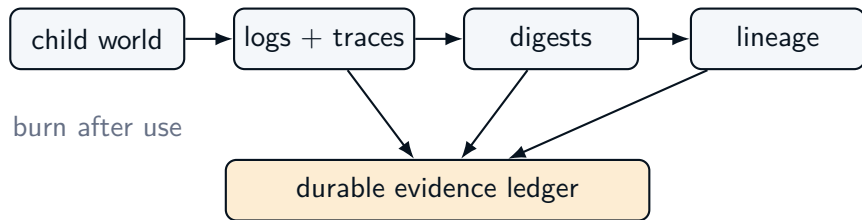
Dangerous warmth

- ▶ sibling-visible writable caches
- ▶ token/session material
- ▶ cross-tenant memory sharing
- ▶ hidden verifier residue

Rule

Pack by trust domain, not by utilization alone.

Evidence must outlive the child



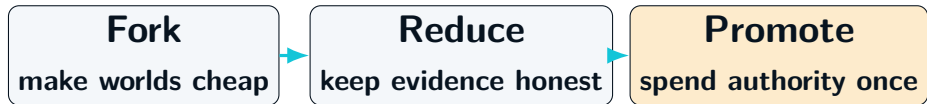
The runner

is disposable. The proof is not.

Computer science translation dictionary

Perspective	Object	Same idea in this talk
Git	branch / merge	child world / promote
Databases	transaction / commit / roll-back	fork / promote / burn
Distributed systems	MapReduce	many workers, structured reduce
Operating systems	fork / checkpoint	executable continuation state
Security	capability	authority is explicit and scoped
Statistics	correlated samples	shared ancestry reduces evidence value
Science	experiment / lab notebook	frozen state, oracle, durable record
RL	rollout / reward	episode, verifier, policy update

Fork, Reduce, Promote: the capability contract



Durable intent. Disposable execution. Singular authority. Deterministic convergence.

Search may be stochastic. **Authority may not be ambient.**

Reduce is not majority vote

A worker should not return only:

score = 0.94

It should return a structured record:

result + precision + evidence IDs +
lineage + method

Why?

- ▶ two siblings may share the same bad root
- ▶ one worker may claim absurd confidence
- ▶ tests may be changed mid-epoch
- ▶ evidence may be missing or stale

When the reducer cannot tell, **abstain** is a valid result.

Four clones do not make four witnesses

$$S_0 \rightarrow \{w_1, w_2, w_3, w_4\}$$

They may share:

- ▶ the same prompt and model
- ▶ the same cached artifact
- ▶ the same hidden bad fixture
- ▶ the same ancestor state

Cold liar

If a worker can claim absurd confidence, it can dominate a reducer even when it is wrong.

Correlation

Execution isolation is not statistical independence.

Fork makes search cheap. It does not manufacture evidence.

Evidence-Aware / Boltzmann MapReduce: Eliaz, arXiv:2607.09689.

Three factories. Same loop.

snapshot -> fork -> search -> verify -> reduce -> promote -> burn

Factory	Search	Oracle / reduce	Scarce tip
Software	candidate patches	tests + evidence lineage	merge to main
RL training	rollout trajectories	reward / verifier	policy checkpoint
HIL / science	sim / hypotheses	measured evidence	robot / wet / paper

The isomorphism

Same runtime contract. Different nouns. The farther right you go, the more expensive a bad promote becomes.

Scientific computing and biology fit the same shape

Cheap search

- ▶ simulation parameters
- ▶ sequence designs
- ▶ pipeline settings
- ▶ mesh / timestep choices
- ▶ generated hypotheses

Scarce truth

- ▶ wet-lab assay
- ▶ clinical review
- ▶ large validated run
- ▶ paper claim
- ▶ production decision

Lesson

The more expensive the oracle, the more valuable safe cheap branching becomes.

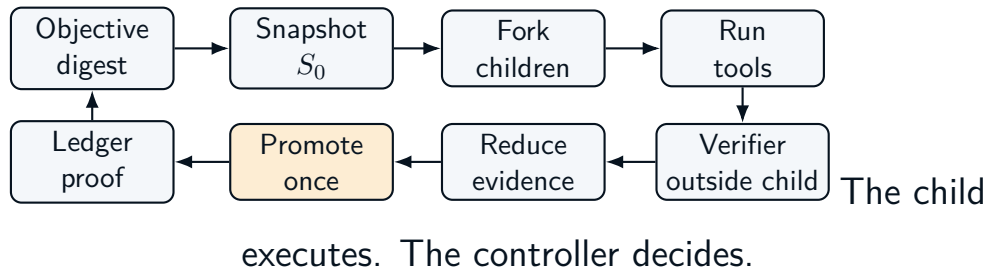
Search dynamics: annealing, without mysticism

Analogy	Runtime meaning
Simulated annealing	Start hot: explore many candidates. Cool: tighten gates.
Metropolis-Hastings	Propose a child move; keep or burn based on evidence.
SGD / Langevin	Local noisy improvement under an external schedule.
RL temperature / KL	Exploration policy belongs to the controller, not the worker.

Claim fence

This is schedule language, not a claim that CI is an equilibrium thermodynamic system.

Architecture blueprint



Epochs and fencing make promotion linearizable

Epoch = frozen experiment contract

- ▶ objective digest O_e
- ▶ parent snapshot S_e
- ▶ verifier identity V_e
- ▶ budget / schedule / policy

Every child carries the epoch

A child created under epoch e may return evidence for e only.

Fencing rule

If the objective, verifier, or trusted parent changes:

$$e \rightarrow e + 1$$

All unfinished children from e become **stale for promotion**.

Linearization point

Promote(candidate, epoch= e)
succeeds exactly once if e is still current.

Search can race. **Promotion must not.**

A research claim needs a benchmark, not a slogan

Question	Measure
How cheap is reset?	time-to-ready, p50/p99 restore/fork latency, rollout throughput
What does sharing cost?	RAM, storage, write amplification, cache hit rate
Is reset faithful?	start-state digest equality, process/file drift, nondeterminism budget
Is isolation real?	cross-episode leakage, credential inheritance, verifier-tamper attempts
Is evidence independent enough?	lineage overlap, pairwise correlation, effective sample size
Does it help the task?	validated candidates/hour, reward distribution, task success

Controlled comparison: same repo / task / verifier

cold build → cached container → restore → fork → model-propose + fork-validate

Win on cost × fidelity × trust – not latency alone.

Failure mode checklist

Dimension	Bad smell	Closure
State Authority	trials contaminate each other child has merge/reward keys	immutable S_0 , burn children remint capabilities; promote outside
Oracle Evidence Correlation Warmth	evaluator is writable by subject green badge without lineage siblings counted as independent cache becomes communication channel	sealed verifier digest durable ledger, digests, traces lineage-aware reduce, abstain pack by trust domain

What this is not

- ▶ Not a claim that all containers are bad.
- ▶ Not a claim that fork gives independent samples.
- ▶ Not a claim that learned world models are obsolete.
- ▶ Not a claim that every real-world service is captured by a snapshot.
- ▶ Not a claim that physics analogies prove thermodynamics of software.

What it is

A capability contract for cheap executable search with separated evidence and authority.

Research agenda for Cambridge SRG

1. **Executable state:** what must a first-class machine value include - FS, memory, process tree, GPU, sockets, clock, external state?
2. **Model vs world:** where is the crossover where learned imagination becomes cheaper than forked executable interaction?
3. **Correlation:** how should reducers price shared ancestry instead of pretending siblings are i.i.d.?
4. **Authority:** how do children inherit enough state to act without inheriting the power to grade or promote themselves?
5. **Warmth:** where can caches live without becoming wires?

Takeaways

1. Software agents need **computers**, not just prompts.
2. Coding RL needs **resettable executable environments**, not only datasets.
3. World models make imagination cheap; forkable sandboxes make interaction cheap.
4. Fork is only safe with reduce, evidence, and singular promote.
5. The runner is disposable. The proof is durable.

Burn the runner. Keep the proof. Fork the machine, not the trust.

Selected sources / claim fence

World models	Hafner et al., <i>Training Agents Inside of Scalable World Models</i> (Dreamer 4), arXiv:2509.24527; Li, <i>Contrastive World Models</i> , arXiv:2609.22175.
Fork systems	Firecracker, NSDI 2020; DeltaBox, arXiv:2605.22781; Shepherd, arXiv:2605.10913.
Training environments	SWE-smith, arXiv:2504.21798; Meta Code World Model (2025); RL-environment vendors.
Evidence-aware reduce	Eliaz, <i>Evidence-Aware MapReduce / Boltzmann MapReduce</i> , arXiv:2607.09689.
Reward integrity	<i>From Rebound to Remedy</i> , arXiv:2604.01476.

Claim fence

A fork removes learned-dynamics error only inside the executable boundary it captures. External services, nondeterminism, hardware, and incomplete tests can still make the world partial.

Optional deep cut: world models plus forks, not either/or

A strong system can use both:

1. learned model proposes promising branches,
2. forked executable worlds ground those branches,
3. reducer compares evidence with lineage,
4. controller promotes once.

Hybrid thesis

Use imagination for breadth. Use executable forks for truth.

Optional deep cut: the minimal API shape

API object	Purpose
Objective.digest	Frozen definition of success for one epoch.
Snapshot.id	Parent world identity and reproducibility key.
Fork(spec)	Creates isolated child with scoped capabilities.
Run(cmd/tool)	Produces observations and artifacts.
Evidence.record	Logs, digests, lineage, method, confidence.
Reduce(records)	Accept / reject / abstain with reasons.
Promote(candidate)	Singular authority boundary.

Optional deep cut: why the title says factory

A factory is not only parallel compute. It is a controlled path from motion to matter.

Motion

- ▶ disposable attempts
- ▶ agent sessions
- ▶ rollout trajectories
- ▶ speculative patches

Matter

- ▶ accepted source snapshot
- ▶ evidence ledger
- ▶ release checkpoint
- ▶ paper / product claim

Create entropy where exploration pays; destroy entropy before promotion.